

APP DEV - 1 PLACEMENT PORTAL APPLICATION V1

PROJECT REPORT

1. STUDENT DETAILS

Name : Kathir Ezhil A
Roll Number : 24f1002369
Email : 24f1002369@ds.study.iitm.ac.in

About Me :

I am currently pursuing the IIT Madras BS Degree in Data Science and Applications alongside a Bachelor's degree in EEE at SSN College of Engineering. I have a strong interest in backend systems, data-driven applications, and full-stack web development, with a CGPA of 9.57 at SSN and 9.5 at the IIT Madras Diploma level.

2. PROJECT DETAILS

Project Title: Placelink - Campus Recruitment Portal

Problem Statement:

Institutes need efficient systems to manage campus recruitment activities involving students, companies, and placement drives. Many institutes rely on spreadsheets or manual processes which create problems such as duplicate applications, difficulty managing approvals, and lack of centralized placement tracking. The objective of this project is to build a web-based Placement Portal Application that allows students, companies, and administrators to interact through role-based functionalities.

Approach:

The system was designed using a Flask backend with modular route structure and SQLite database. The application implements role-based access control for Admin, Company, and Student users. The workflow includes:

1. Companies register and await admin approval.
 2. Approved companies create placement drives.
 3. Admin approves drives before they become visible to students.
 4. Students apply for drives based on eligibility.
 5. Companies manage applications and update status (shortlisted / selected / rejected).
 6. Admin monitors overall placement analytics.
-

3. AI/LLM DECLARATION

The approximate AI usage is 15–20%, limited to assistance and learning support.All final code implementation, debugging, integration, and testing were done manually.AI assistance tools (ChatGPT) were used during the development process primarily for debugging suggestions ,UI design improvements, code structuring suggestions, documentation formatting

Combined AI Usage Breakdown Table

Component / Tool	Usage %
Flask Backend (Routing & Structure)	6%
SQLite Database Queries	4%
HTML Templates + Jinja2 Rendering	8%
Bootstrap / CSS Styling	10%
Authentication (Login / Role Management)	5%
Admin Management Logic	3%
Student Application Workflow	6%
Validation (Client + Server Side)	3%
Charts & Analytics (Chart.js Integration)	4%

Category Level AI Usage Summary

Category	Total Usage %
Backend (Flask + Database)	20%
Frontend (HTML + Jinja2 Templates)	18%
UI Styling / Layout (Bootstrap)	10%
Authentication & CRUD Logic	8%
Analytics & Additional Features	5%

4. TECHNOLOGIES AND FRAMEWORKS USED

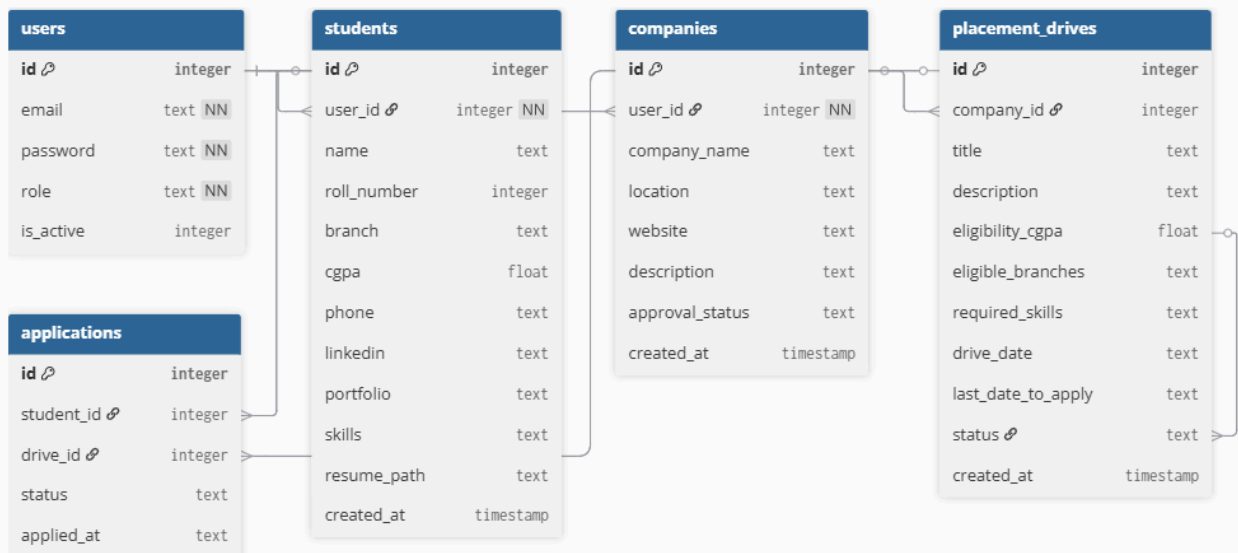
Technology / Library	Purpose
Flask	Backend web framework
Jinja2	Dynamic HTML template rendering
Bootstrap 5	UI styling and responsive layout
Chart.js	Data visualization for placement analytics
SQLite	Database storage
Werkzeug Security	Password hashing
Python	Backend logic and application development

5. DATABASE SCHEMA AND TABLES

1. **User** — stores authentication details (*id, email, password, role, is_active*)
2. **Student** — stores student profile details (*id, user_id, name, roll_number, branch, cgpa, skills, resume_path*)
3. **Company** — stores company profile details (*id, user_id, company_name, location, website, description, approval_status*)
4. **PlacementDrive** — stores placement drive details (*id, company_id, title, description, eligibility_cgpa, eligible_branches, required_skills, drive_date, last_date_to_apply, status*)
5. **Application** — stores student applications to drives (*id, student_id, drive_id, status, applied_at*)

Relationships:

- One-to-One → **User** → **Student**
- One-to-One → **User** → **Company**
- One-to-Many → **Company** → **PlacementDrive**
- One-to-Many → **Student** → **Application**
- One-to-Many → **PlacementDrive** → **Application**



6. ARCHITETURE AND FEATURES

```
placement_portal/
├── app.py                # Main application entry point
├── database.py           # Database configuration & models
├── load_data.py          # Script for seeding initial data
├── placement.db          # SQLite database file
├── README.md            # Project documentation
├── routes/              # Blueprint-based routing modules
│   ├── auth_routes.py   # Authentication & Session logic
│   ├── student_routes.py # Student dashboard & profile
│   ├── company_routes.py # Job postings & recruitment
│   └── admin_routes.py  # Oversight & approval controls
├── utils/               # Utility functions & helpers
│   └── decorators.py     # Custom decorators (e.g., login_required)
├── templates/           # Jinja2 HTML templates
│   ├── admin/           # Admin-specific views
│   ├── company/         # Recruiter-specific views
│   ├── student/         # Applicant-specific views
│   ├── base.html        # Master layout template
│   ├── login.html       # Login page
│   ├── register.html    # Registration page
│   └── home.html        # Landing/Index page
├── static/              # Static assets
│   ├── icons/           # UI icons
│   └── images/          # Logos
└── uploads/             # File uploads
    └── resumes/         # Stored PDF/DOCX resumes
```

1. System Foundations: Security & Access

The platform is built on a robust security framework to ensure data integrity and controlled access.

Core Security Feature	Technical Implementation
Authentication	Multi-user login system with distinct session management for all roles.
Data Protection	Industry-standard password hashing utilizing the Werkzeug Security library.
Access Control	Strict Role-Based Access Control (RBAC) to enforce permission boundaries.
Account Seeding	Administrative credentials are pre-provisioned within the database for immediate oversight.

2. User Role Functionalities

Admin Module

The admin manages the overall functioning of the portal. This includes approving or rejecting company registrations and placement drives, monitoring users, and viewing platform statistics such as the total number of students, companies, drives, and applications.

Company Module

Companies can register and create their profiles after admin approval. They can create and manage placement drives, view student applications, shortlist candidates, and update the recruitment status.

Student Module

Students can build their profiles, upload resumes, and explore available placement drives. They can apply to eligible drives and track their application status and placement history through their dashboard.

3. Advanced Logic & System Intelligence

Feature	Description & Logic
Smart Recommendation	Automated eligibility engine matching student CGPA and skills to drive requirements.
Profile Tracking	Visual progress indicators showing completion percentage to drive user engagement.
Placement Analytics	Interactive data visualization using Chart.js for application trends and distribution.
Integrity Guard	Built-in duplicate prevention to ensure one application per student per drive.
Document Vault	Secure filesystem-based resume management system.
Deadline Awareness	Time-sensitive application locking based on pre-defined drive expiration dates.

8. Video Presentation

Drive Link:

[Click here to see the video presentation](#)
